

Kadidia Ouedraogo

Prof. Yanilda Peralta Ramos

CIS: 345

05/13/2016

Final Project

Lottery Scheduler Implementation in xv6

Overview

In this project, I implemented a Lottery Scheduler in the xv6 operating system. Each process is assigned a number of tickets, and at each time slice the scheduler randomly picks a winning ticket. The process that holds the winning ticket gets to run. Processes with more tickets have a higher probability of being selected

Learning Objective :

- How the xv6 kernel is structured
- How CPU schedulers work
- How to modify an existing scheduler
- How to analyze performance using graphs

-System Calls Implemented

settickets(int number) Sets the number of lottery tickets for the current process. Returns 0 if successful and -1 if the number is less than 1.

getpinfo(struct pstat) Fills a pstat structure with information about all processes including PID, tickets, and ticks. Returns 0 on success and -1 on failure.

- pstat Structure

The pstat structure was provided and not modified. It contains four arrays:

-inuse[] , whether process slot is in use

-tickets[] , number of tickets per process

-pid[] , process ID

-ticks[] , how many times process was scheduled

- Implementation Requirements

-Added tickets and ticks fields to proc structure in proc.h

-Default tickets set to 1 for every new process in allocproc()

Child processes inherit tickets from parent in fork()

Random number generator added to select winning ticket

Ticks field incremented each time a process is scheduled

Pointer validation added in getpinfo using copyout

- Scheduler Logic The lottery scheduler works in 5 steps:

Calculate total tickets of all RUNNABLE processes

Generate random number within total ticket range

Walk process list to find the winning process

Run the winning process

Increment its tick count

-Files created and Modified

-kernel/pstat.h: I created new file because it needed to defines pstat structure to store process statistics.

```

GNU nano 2.9.4
#ifndef _PSTAT_H_
#define _PSTAT_H_

#include "param.h"

struct pstat {
    int inuse[NPROC];
    int tickets[NPROC];
    int pid[NPROC];
    int ticks[NPROC];
};

#endif

```

-kernel/proc.h : I added tickets and ticks fields to struct proc: to allow every process to track how many tickets it has and how many time it ran.

```

int tickets;           // lottery tickets
int ticks;             // times scheduled

```

-kernel/proc.c : I added a random number generator at the top, lottery scheduler, default tickets, child inheritance. I set code in xv6 because xv6 has no built - in random function, and needed for lottery selection.

```

unsigned long randstate = 1;
unsigned int rand(void) {
    randstate = randstate * 1664525 + 10139004223;
    return randstate;
}

```

-Set default tickets in allocproc(): This allows every new process to start with 1 ticket by default.

```

p->state = USED;
p->tickets = 1;
p->ticks = 0;

```

-Child inherits tickets in Fork(): This allows Child to get same tickets as parent when a process forks.

```

np->tickets = p->tickets;

```

-Replace scheduler with lottery scheduler: this is the core lottery scheduler this allows to pick winner randomly based on tickets.

```

void
scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();

    c->proc = 0;
    for(;;){
        intr_on();

        int total = 0;
        for(p = proc; p < &proc[NPROC]; p++){
            acquire(&p->lock);
            if(p->state == RUNNABLE)
                total += p->tickets;
            release(&p->lock);
        }

        int winner = 0;
        if(total > 0)
            winner = rand() % total;
        int counter = 0;
        for(p = proc; p < &proc[NPROC]; p++){
            acquire(&p->lock);
            if(p->state == RUNNABLE){
                counter += p->tickets;
                if(counter > winner){
                    p->state = RUNNING;
                    p->ticks++;
                    if(p->pid > 3){
                        printf("tick:%d pid:%d tickets:%d\n", p->ticks, p->pid, p->tickets);
                    }
                    c->proc = p;
                    swtch(&c->context, &p->context);
                    c->proc = 0;
                    release(&p->lock);

```

```

                release(&p->lock);
                break;
            }
        }
        release(&p->lock);
    }
}

```

-kernel/syscall.h : I added syscall numbers 28 and 29 because every system call needs a unique number.

```
#define SYS_settickets 28  
#define SYS_getpinfo 29
```

-kernel/syscall.c: I added extern declarations and table entries, this allows to register the new system calls so kernel knows about them.

```
extern uint64 sys_settickets(void);  
extern uint64 sys_getpinfo(void);
```

Added to syscall table:

```
[SYS_settickets] sys_settickets,  
[SYS_getpinfo] sys_getpinfo,  
};
```

-kernel/sysproc.c : I implemented sys_settickets and sys_getpinfo

I added two new functions: This implements the actual system call logic

```
#include "pstat.h"  
  
extern struct proc proc[];
```

```

uint64
sys_settickets(void)
{
    int n;
    argint(0, &n);
    if(n < 1) return -1;
    myproc()->tickets = n;
    return 0;
}

uint64
sys_getpinfo(void)
{
    uint64 addr;
    argaddr(0, &addr);
    struct pstat ps;
    struct proc *p;
    int i = 0;
    for(p = proc; p < &proc[NPROC]; p++){
        acquire(&p->lock);
        ps.inuse[i] = (p->state != UNUSED);
        ps.tickets[i] = p->tickets;
        ps.pid[i] = p->pid;
        ps.ticks[i] = p->ticks;
        release(&p->lock);
        i++;
    }
    if(copyout(myproc()->pagetable, addr, (char*)&ps, sizeof(ps)) < 0)
        return -1;
    return 0;
}

```

-kernel/defs.h : I added pstat declaration

Forward declaration so other files know pstat exists.

```

struct pstat;

```

user/user.h : I added function declarations because the user programs need to know these system calls exist.

```

struct pstat;

```

```

int settickets(int);
int getpinfo(struct pstat*);

```


-user/usys.pl : I added syscall entries to generate the assembly stub code that connects user calls to kernel.

```
entry("settickets");  
entry("getpinfo");
```

-Makefile : I added lottery_test to UPROGS because it tells the build system to compile lottery_test program.

```
$U/_lottery_test
```

-user/lottery_test.c : I Created new file test program that :

-Forks 3 processes

-Sets tickets 30,20,10

-Runs busy loops

-Calls getpinfo to get stats

-Prints results.

```

#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/pstat.h"

int
main(void)
{
    struct pstat st;
    int pid1, pid2, pid3;

    pid1 = fork();
    if(pid1 == 0){
        settickets(30);
        while(1){
            for(volatile int i = 0; i < 100000; i++);
        }
    }

    pid2 = fork();
    if(pid2 == 0){
        settickets(20);
        while(1){
            for(volatile int i = 0; i < 100000; i++);
        }
    }

    pid3 = fork();
    if(pid3 == 0){
        settickets(10);
        while(1){
            for(volatile int i = 0; i < 100000; i++);
        }
    }
}

```

-Graph Analysis

Bar Chart Analysis:

The bar chart shows the average number of ticks each process received over 6 runs of the lottery scheduler test. Three processes were created with a ticket ratio of 3:2:1 (30, 20, and 10 tickets respectively).

Process A (30 tickets) received an average of 311.5 ticks, the highest because it holds the most tickets (50% of total tickets)

Process B (20 tickets) received an average of 183.2 ticks, the second highest because it holds 33% of total tickets

Process C (10 tickets) received an average of 92.3 ticks, the lowest because it holds only 17% of total tickets

This confirms that the lottery scheduler correctly allocates CPU time proportionally to the number of tickets each process holds.

Test Results: Bar Chart: The bar chart shows average ticks over 6 runs

Process A (30 tickets) = 311.5 average ticks (50% CPU)

Process B (20 tickets) = 183.2 average ticks (33% CPU)

Process C (10 tickets) = 92.3 average ticks (17% CPU)

Scheduler Behavior Analysis over 1 time run :

-1st place: Process A (PID4) 30 tickets, 317 ticks (53% CPU)

-2nd place: Process B (PID5) 20 tickets, 178 ticks (30% CPU)

-3rd place: Process C (PID6) 10 tickets, 94 ticks (16% CPU)

Gantt Chart Analysis:

The Gantt chart shows the scheduling order over 30 time slots. Each row represents one time slot and shows which process was selected by the lottery scheduler. The chart clearly shows:

-Process A (blue) appears most frequently

-Process B (orange) appears second most frequently

-Process C (grey) appears least frequently

The representation shows that the lottery scheduler selects processes randomly but proportionally to their ticket count, following the expected 3:2:1 distribution.

Explanation:

-The lottery scheduler works as follows:

-Each process is assigned a number of tickets

-At each time slice the scheduler counts total tickets of all RUNNABLE processes

-A random number is generated within the total ticket range

-The scheduler walks through the process list adding up tickets until it reaches the winning number

-The winning process runs and its tick count is incremented

-Processes with more tickets have a higher probability of winning

Bar Chart: The bar chart shows average ticks over 6 runs

```

riscv64-linux-gnu-objdump -t user/_wc | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/wc.sym
riscv64-linux-gnu-ld -z max-page-size=4096 -T user/user.ld -o user/_zombie user/zombie.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-linux-gnu-objdump -S user/_zombie > user/zombie.asm
riscv64-linux-gnu-objdump -t user/_zombie | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/zombie.sym
riscv64-linux-gnu-ld -z max-page-size=4096 -T user/user.ld -o user/_logstress user/logstress.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-linux-gnu-objdump -S user/_logstress > user/logstress.asm
riscv64-linux-gnu-objdump -t user/_logstress | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/logstress.sym
riscv64-linux-gnu-ld -z max-page-size=4096 -T user/user.ld -o user/_forphan user/forphan.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-linux-gnu-objdump -S user/_forphan > user/forphan.asm
riscv64-linux-gnu-objdump -t user/_forphan | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/forphan.sym
riscv64-linux-gnu-ld -z max-page-size=4096 -T user/user.ld -o user/_dorphan user/dorphan.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-linux-gnu-objdump -S user/_dorphan > user/dorphan.asm
riscv64-linux-gnu-objdump -t user/_dorphan | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/dorphan.sym
riscv64-linux-gnu-ld -z max-page-size=4096 -T user/user.ld -o user/_getprocs user/getprocs.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-linux-gnu-objdump -S user/_getprocs > user/getprocs.asm
riscv64-linux-gnu-objdump -t user/_getprocs | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/getprocs.sym
riscv64-linux-gnu-ld -z max-page-size=4096 -T user/user.ld -o user/_test_glock user/test_glock.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-linux-gnu-objdump -S user/_test_glock > user/test_glock.asm
riscv64-linux-gnu-objdump -t user/_test_glock | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/test_glock.sym
riscv64-linux-gnu-ld -z max-page-size=4096 -T user/user.ld -o user/_test_sem user/test_sem.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-linux-gnu-objdump -S user/_test_sem > user/test_sem.asm
riscv64-linux-gnu-objdump -t user/_test_sem | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/test_sem.sym
riscv64-linux-gnu-ld -z max-page-size=4096 -T user/user.ld -o user/_lottery_test user/lottery_test.o user/ulib.o user/usys.o user/printf.o user/umalloc.o
riscv64-linux-gnu-objdump -S user/_lottery_test > user/lottery_test.asm
riscv64-linux-gnu-objdump -t user/_lottery_test | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > user/lottery_test.sym
mkfs/mkfs fs.img README user/_cat user/_echo user/_forktest user/_grep user/_init user/_kill user/_ln user/_ls user/_mkdir user/_rm user/_sh user/_str
essfs user/_usertests user/_grind user/_wc user/_zombie user/_logstress user/_forphan user/_dorphan user/_getprocs user/_test_glock user/_test_sem use
r/_lottery_test
nmeta 47 (boot, super, log blocks 31, inode blocks 13, bitmap blocks 1) blocks 1953 total 2000
ballocc: first 1099 blocks have been allocated
ballocc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 1 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img
,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

init: starting sh
$ lottery_test

```

```

tick:303 pid:4 tickets:30
tick:168 pid:5 tickets:20
tick:304 pid:4 tickets:30
tick:93 pid:6 tickets:10
tick:305 pid:4 tickets:30
tick:306 pid:4 tickets:30
tick:169 pid:5 tickets:20
tick:170 pid:5 tickets:20
tick:171 pid:5 tickets:20
tick:94 pid:6 tickets:10
tick:172 pid:5 tickets:20
tick:173 pid:5 tickets:20
tick:307 pid:4 tickets:30
tick:95 pid:6 tickets:10
tick:308 pid:4 tickets:30
tick:309 pid:4 tickets:30
tick:96 pid:6 tickets:10
tick:310 pid:4 tickets:30
tick:174 pid:5 tickets:20
tick:311 pid:4 tickets:30
tick:312 pid:4 tickets:30
tick:313 pid:4 tickets:30
tick:314 pid:4 tickets:30
tick:315 pid:4 tickets:30
tick:175 pid:5 tickets:20
tick:176 pid:5 tickets:20
tick:177 pid:5 tickets:20
tick:97 pid:6 tickets:10
tick:316 pid:4 tickets:30
tick:178 pid:5 tickets:20
tick:317 pid:4 tickets:30
PID Tickets Ticks
4 30 317
5 20 178
6 10 97
tick:179 pid:5 tickets:20
tick:318 pid:4 tickets:30
tick:98 pid:6 tickets:10
$

```

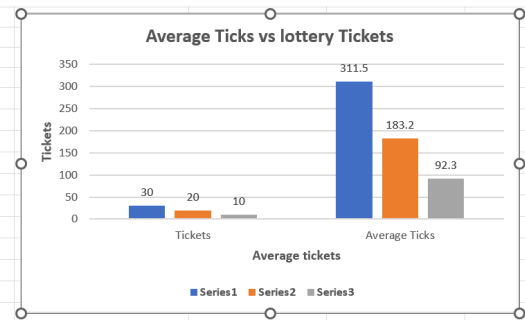
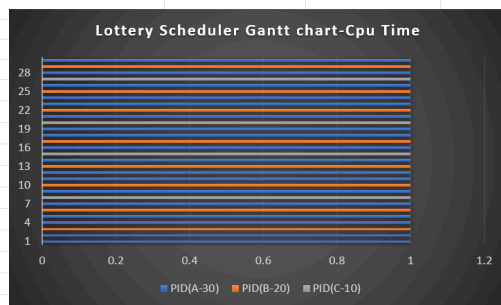
Time Slot	PID(A-30)	PID(B-20)	PID(C-10)
1	1	0	0
2	1	0	0
3	0	1	0
4	1	0	0
5	1	0	0
6	0	1	0
7	1	0	0
8	0	0	1
9	1	0	0
10	0	1	0
11	1	0	0
12	1	0	0
13	0	1	0
14	1	0	0
15	0	0	1
16	1	0	0
17	0	1	0
18	1	0	0
19	1	0	0
20	0	0	1
21	1	0	0
22	0	1	0
23	1	0	0
24	1	0	0
25	0	1	0
26	1	0	0
27	0	0	1
28	1	0	0
29	0	1	0
30	1	0	0

RUN	TICKETS	TICKS
1	30	314
1	20	178
1	10	94
2	30	317
2	20	178
2	10	97
3	30	301
3	20	197
3	10	90
4	30	318
4	20	178
4	10	94
5	30	301
5	20	190
5	10	85
6	30	318
6	20	178
6	10	94

Screenshot of the average ticks vs lottery tickets

Tickets	Average Ticks
30	311.5
20	183.2
10	92.3

Graphs screenshots



-Commands used:

```
kadidia-ouedraogo@kadidia-ouedraogo-VirtualBox:~/xv6-riscv$ nano ~/xv6-riscv/kernel/pstat.h
kadidia-ouedraogo@kadidia-ouedraogo-VirtualBox:~/xv6-riscv$ nano ~/xv6-riscv/kernel/proc.h
kadidia-ouedraogo@kadidia-ouedraogo-VirtualBox:~/xv6-riscv$ nano ~/xv6-riscv/kernel/proc.c
kadidia-ouedraogo@kadidia-ouedraogo-VirtualBox:~/xv6-riscv$ nano ~/xv6-riscv/kernel/syscall.h
kadidia-ouedraogo@kadidia-ouedraogo-VirtualBox:~/xv6-riscv$ nano ~/xv6-riscv/kernel/syscall.c
kadidia-ouedraogo@kadidia-ouedraogo-VirtualBox:~/xv6-riscv$ nano ~/xv6-riscv/kernel/sysproc.c
kadidia-ouedraogo@kadidia-ouedraogo-VirtualBox:~/xv6-riscv$ nano ~/xv6-riscv/kernel/defs.h
kadidia-ouedraogo@kadidia-ouedraogo-VirtualBox:~/xv6-riscv$ nano ~/xv6-riscv/user/user.h
kadidia-ouedraogo@kadidia-ouedraogo-VirtualBox:~/xv6-riscv$ nano ~/xv6-riscv/user/usys.pl
kadidia-ouedraogo@kadidia-ouedraogo-VirtualBox:~/xv6-riscv$ nano ~/xv6-riscv/user/usys.pl
kadidia-ouedraogo@kadidia-ouedraogo-VirtualBox:~/xv6-riscv$ nano ~/xv6-riscv/Makefile
kadidia-ouedraogo@kadidia-ouedraogo-VirtualBox:~/xv6-riscv$ nano ~/xv6-riscv/user/lottery_test.c
kadidia-ouedraogo@kadidia-ouedraogo-VirtualBox:~/xv6-riscv$
```